

UrbanHeat Accra — Machine-Learning-Based Urban Heat Risk Prediction & Mitigation-Simulation Dashboard

Student Name: Emmanuel Owusu

Student ID: 22425075

Institution: [TO BE PROVIDED BY STUDENT]

Department: [TO BE PROVIDED BY STUDENT]

Programme: [TO BE PROVIDED BY STUDENT]

Supervisor: [TO BE PROVIDED BY STUDENT]

Date: 15 August 2026

Table of Contents

1. Project Title
2. Background
3. Problem Statement
4. Aim
5. Objectives
6. Stakeholders
7. Requirements Analysis
8. Functional Requirements
9. Non-Functional Requirements
10. Software Requirements Specification (SRS)
11. Requirements Prioritisation
12. Software Effort Estimation
13. System Analysis
14. System Design
15. Architecture
16. Use Case Analysis
17. Database / ER Design
18. UML Diagrams
19. Implementation
20. Frontend
21. Backend
22. Database
23. Authentication & Authorisation
24. Validation
25. Security
26. Testing
27. Test Results

- 28. Technical Debt
- 29. Technical Debt Repayment Plan
- 30. Deployment
- 31. User Manual
- 32. Maintenance Strategy
- 33. Future Evolution
- 34. Limitations
- 35. Conclusion
- 36. References
- 37. Final Compliance Checklist

List of Figures

Figure	Description
Figure 1	System Architecture Diagram
Figure 2	Component Diagram
Figure 3	Entity-Relationship Diagram
Figure 4	Use Case Diagram
Figure 5	Sequence Diagram — View Risk Map & Drill Into Location
Figure 6	Sequence Diagram — Run Mitigation Simulation
Figure 7	Activity Diagram — User Workflow
Figure 8	Map Dashboard Screenshot
Figure 9	Location Detail Drawer Screenshot
Figure 10	Mitigation Simulation Screenshot
Figure 11	Data Explorer Screenshot
Figure 12	Risk Predictor Tool Screenshot
Figure 13	Deployment Architecture

List of Tables

Table	Description
Table 1	Stakeholder Analysis
Table 2	Functional Requirements
Table 3	Non-Functional Requirements
Table 4	Requirements Prioritisation (MoSCoW)
Table 5	Use Case Specifications
Table 6	Database Schema — Locations Table
Table 7	API Endpoint Specification
Table 8	Test Cases and Results
Table 9	Technical Debt Register
Table 10	Technical Debt Repayment Roadmap
Table 11	Requirements Traceability Matrix

1. Project Title

Machine-Learning-Based Urban Heat Risk Prediction and Mitigation-Simulation Dashboard for Accra, Ghana (UrbanHeat Accra)

2. Background

Urban heat islands (UHI) represent one of the most significant environmental challenges facing rapidly urbanising cities in tropical developing countries. Accra, the capital of Ghana, with a metropolitan population exceeding 4 million, experiences intensifying urban heat effects driven by rapid unplanned development, declining green cover, and increasing impervious surface areas. The intersection of high built-up density, limited vegetation, and coastal humidity creates localised heat zones that disproportionately affect vulnerable populations.

Machine learning offers a transformative approach to understanding and predicting urban heat vulnerability at a granular spatial level. By analysing multispectral environmental indicators — including vegetation indices (NDVI), built-up surface density, proximity to green spaces, and terrain elevation — predictive models can generate location-specific heat risk scores that support evidence-based urban planning decisions.

This project develops a full-stack web application that combines a trained Random Forest regression model with an interactive geospatial dashboard, enabling urban planners and environmental officers to:

- Visualise heat vulnerability across Accra's neighbourhoods,
- Understand the driving environmental factors behind each location's risk level,
- Simulate the potential impact of vegetation-based mitigation interventions.

The system is designed within a 48-hour intensive development constraint, targeting a Minimum Viable Product (MVP) that demonstrates the technical feasibility and practical value of AI-assisted climate resilience planning.

3. Problem Statement

Urban planners and environmental officers in Greater Accra currently lack accessible, location-specific tools to identify, understand, and respond to differential heat vulnerability across the metropolitan area. Existing climate data is typically available only at aggregate city-wide or regional scales, making it difficult to prioritise interventions at the neighbourhood level where they would have the greatest impact.

There is no readily available system that:

1. Provides granular, location-level heat risk predictions based on measurable environmental features,
2. Explains *why* specific locations are at higher risk in plain, non-technical language,
3. Allows planners to simulate the effect of proposed greening interventions before committing resources.

Without such a tool, planning decisions regarding tree planting, green infrastructure, and

urban canopy expansion are made without quantitative spatial evidence, leading to suboptimal resource allocation and missed opportunities for targeted climate adaptation.

4. Aim

To design, develop, and deploy a machine-learning-powered web dashboard that predicts urban heat risk across Accra, Ghana, explains the contributing factors in human-readable terms, and enables planners to simulate vegetation-based mitigation strategies at the individual location level.

5. Objectives

1. **O1:** Train a Random Forest regression model on environmental features (NDVI, built-up density, distance to green space, elevation) to predict heat risk scores (0–100) for locations across Accra.
 2. **O2:** Build a RESTful API (FastAPI) that serves model predictions, feature explanations, and intervention simulations through validated endpoints.
 3. **O3:** Develop an interactive frontend dashboard (React + Leaflet.js) that displays heat risk data on a colour-coded map, with drill-down location detail panels.
 4. **O4:** Implement a mitigation simulation feature that allows users to model the effect of increased vegetation cover on predicted heat risk.
 5. **O5:** Implement explainability features that present the top contributing factors for each location's risk score in plain language.
 6. **O6:** Deploy the full-stack application to cloud platforms (Render for backend, Vercel for frontend) for public accessibility.
 7. **O7:** Produce comprehensive project documentation suitable for academic submission.
-

6. Stakeholders

Table 1: Stakeholder Analysis

Stakeholder	Role	Interest	Impact
Urban Planners (Primary Users)	Use the dashboard to identify heat-vulnerable areas and prioritise greening interventions	High — direct operational benefit from heat risk maps and simulation results	High — primary users whose planning decisions are informed by system outputs
Environmental Officers	Monitor environmental risk indicators across the metropolitan area	High — need spatial understanding of environmental vulnerability patterns	Medium — use data for policy and compliance, not direct planning
City Administration	Decision-makers for resource allocation to climate adaptation projects	Medium — require evidence to justify infrastructure spending	High — budget and policy decisions depend on reliable evidence

Residents of High-Risk Areas	Indirectly affected by planning decisions made using the tool	High — health and comfort directly impacted by urban heat	Low — not direct users, but ultimate beneficiaries
Academic Supervisor / Examiner	Evaluate the project's technical quality, documentation, and learning outcomes	High — assessment of the project	High — determines academic outcome
Developer (Student)	Design, implement, test, and document the system	High — responsible for all aspects of delivery	High — sole developer within the 48-hour constraint

7. Requirements Analysis

Requirements were derived from the project's aim, stakeholder analysis, and the technical constraints of a 48-hour MVP development window. The analysis followed a structured approach:

- 1. Stakeholder interviews (simulated):** Identified that urban planners need plain-language explanations, not raw model internals; need before/after comparisons for greening interventions; and need map-based spatial exploration.
- 2. Domain research:** Reviewed existing urban heat island tools and climate resilience dashboards to identify common patterns and gaps.
- 3. Technical feasibility assessment:** Evaluated available technology stacks against the 48-hour development constraint, selecting Python (FastAPI, scikit-learn, SQLite) and React (Vite, Tailwind CSS, Leaflet) for maximum developer productivity.
- 4. MoSCoW prioritisation:** Classified requirements as Must, Should, Could, or Won't to scope the MVP appropriately.

8. Functional Requirements

Table 2: Functional Requirements

ID	Requirement	Priority	Status
FR1	The system shall display all monitored locations on an interactive map, colour-coded by heat risk category (Low, Moderate, High, Severe)	Must	Implemented
FR2	The system shall allow users to select a location (by clicking a map pin or searching by name) and view its heat risk score (0–100) and category	Must	Implemented
FR3	The system shall display the top contributing environmental factors for each location's risk score, with visual importance ranking	Must	Implemented
FR4	The system shall allow users to simulate a vegetation increase (user-defined percentage) on any location and display the before/after risk comparison	Must	Implemented

FR5	The system shall expose a RESTful API with validated endpoints for locations, predictions, explanations, and simulations	Must	Implemented
FR6	The system shall provide a data explorer view with a searchable, sortable, and exportable table of all locations and their features	Should	Implemented
FR7	The system shall allow users to filter map pins by risk category (All / Low / Moderate / High / Severe)	Should	Implemented
FR8	The system shall provide a standalone risk prediction tool where users can input custom environmental features and receive a predicted risk score	Should	Implemented
FR9	The system shall provide an admin interface for managing location data	Won't	Descoped — out of MVP scope

9. Non-Functional Requirements

Table 3: Non-Functional Requirements

ID	Requirement	Category	Target	Status
NFR1	Prediction responses shall complete within 1 second under normal conditions	Performance	< 1s response time for /api/predict	Met — model loaded at startup, inference < 100ms typical
NFR2	The interface shall be usable by planners with no machine learning background	Usability	Plain-language labels, no raw model internals exposed	Met — all UI labels use domain terms
NFR3	The system shall work correctly across Chrome, Firefox, and Edge browsers	Compatibility	Cross-browser support	Met — standard React/Leaflet, no browser-specific APIs
NFR4	The frontend shall be responsive across desktop ($\geq 1024\text{px}$), tablet ($600\text{--}1023\text{px}$), and mobile ($< 600\text{px}$) viewports	Responsiveness	Responsive layout at all breakpoints	Partially met — functional on mobile, optimised for desktop
NFR5	The system shall be deployable without external database services (SQLite default, Postgres upgrade path)	Portability	Zero-config database	Met — SQLite with Postgres upgrade path via DATABASE_URL
NFR6	All user inputs shall be validated server-side with clear error messages; no internal server errors shall be exposed to users	Security / Validation	422 on invalid input, 500 never leaks stack traces	Met — Pydantic validation + global exception handler
NFR7	Every prediction score shall be accompanied by explainability information (top contributing factors)	Explainability	Factor importance alongside every score	Met — /api/explain endpoint + UI factor bars

NFR8 The API shall provide a health check endpoint for deployment monitoring	Reliability	/api/health returns {"status": "ok"}	Met — implemented
--	-------------	--------------------------------------	-------------------

10. Software Requirements Specification (SRS)

A standalone, detailed SRS document is provided separately as *SRS.pdf*. This section provides a summary.

10.1 Purpose

This SRS defines the functional and non-functional requirements for the UrbanHeat Accra system — a web-based dashboard for urban heat risk prediction, explainability, and mitigation simulation.

10.2 Scope

The system covers heat risk visualisation, single-location drill-down with factor explanations, vegetation-based mitigation simulation, data export, and direct ML prediction for custom parameters. It does not cover real-time satellite data ingestion, multi-city support, authentication, or administrative data management.

10.3 Product Perspective

UrbanHeat Accra is a standalone web application. It does not integrate with existing municipal IT systems. It uses a pre-trained machine learning model and a pre-seeded dataset of 250 locations across the Accra metropolitan area.

10.4 System Interfaces

- **Frontend ↔ Backend:** RESTful JSON API over HTTPS
- **Backend ↔ Database:** SQLAlchemy ORM over SQLite (or PostgreSQL)
- **Backend ↔ ML Model:** joblib-loaded scikit-learn model file (heat_risk_model.pkl)

10.5 Acceptance Criteria

1. All Must-priority functional requirements (FR1–FR5) are implemented and testable.
 2. API endpoints return correct responses for valid inputs and appropriate errors for invalid inputs.
 3. The map displays colour-coded location pins with correct risk categories.
 4. The simulation produces before/after risk comparisons that directionally make sense (more vegetation → lower or equal risk).
 5. The system deploys successfully to Render (backend) and Vercel (frontend).
-

11. Requirements Prioritisation

Requirements were prioritised using the MoSCoW framework:

Table 4: Requirements Prioritisation (MoSCoW)

Priority	Requirements	Rationale
Must	FR1, FR2, FR3, FR4, FR5, NFR1, NFR5, NFR6, NFR7, NFR8	Core value proposition: map, predict, explain, simulate. Without these, the system has no demonstrable purpose.
Should	FR6, FR7, FR8, NFR2, NFR3, NFR4	Enhance usability and completeness but are not required for the core prediction-explanation-simulation workflow.
Could	Custom slider for vegetation delta, per-instance SHAP values, data pagination	Nice-to-have refinements that improve UX but do not change core functionality.
Won't	FR9 (admin panel), real-time satellite data, multi-city support, authentication, user accounts	Out of scope for a 48-hour MVP. Documented in Technical Debt for future versions.

12. Software Effort Estimation

12.1 Estimation Approach

Given the 48-hour constraint, a simplified Function Point Analysis (FPA) was used to estimate effort distribution across major components.

12.2 Effort Breakdown

Component	Estimated Effort (Hours)	Actual Effort (Approx.)
Requirements analysis & design	4	3
ML model training & validation	3	3
Backend API development	8	8
Database schema & seeding	2	2
Frontend React components	16	18
Map integration (Leaflet)	4	4
Testing & debugging	4	4
Deployment configuration	3	3
Documentation	4	3
Total	48	48

12.3 Observations

The frontend consumed the largest share of effort (37%), consistent with the rich interactive features required (map, drawer, simulation, data table, predictor modal). The backend was relatively straightforward due to FastAPI's automated validation and documentation features.

13. System Analysis

13.1 Current System (As-Is)

Currently, urban planners in Accra rely on:

- Aggregate climate reports at city or regional scale
- Manual inspection and experience-based assessment
- No spatial, location-specific heat risk quantification tool
- No what-if simulation capability for greening interventions

13.2 Proposed System (To-Be)

UrbanHeat Accra introduces:

- **Spatial heat risk quantification** at the individual location level using machine learning
- **Automated factor explanation** identifying why each location is at risk
- **Interactive simulation** allowing planners to test greening interventions before committing resources
- **Data export** for integration with existing planning workflows

13.3 Feasibility Analysis

Feasibility	Assessment
Technical	Feasible — all technologies (FastAPI, React, scikit-learn, Leaflet) are mature and well-documented
Economic	Feasible — uses entirely open-source software; hosting costs minimal (free tiers on Render/Vercel)
Operational	Feasible — browser-based, no installation required, designed for non-technical users
Schedule	Feasible within 48 hours with MoSCoW scoping to MVP

14. System Design

14.1 Design Philosophy

The system follows a **clean separation of concerns** architecture:

- **Presentation Layer:** React SPA responsible for all user interaction and rendering
- **API Layer:** FastAPI RESTful service responsible for business logic, validation, and model inference
- **Data Layer:** SQLite database accessed through SQLAlchemy ORM
- **ML Layer:** Pre-trained scikit-learn model loaded at startup for stateless inference

14.2 Design Patterns Used

- **Repository Pattern (implicit):** SQLAlchemy session management via FastAPI dependency injection
- **Model-View-Controller (adapted):** FastAPI endpoints (Controller) → SQLAlchemy models (Model) → React components (View)
- **Component-Based Architecture:** React frontend decomposed into focused, reusable components (Sidebar, Header, MapView, LocationDrawer, DataExplorer, PredictModal,

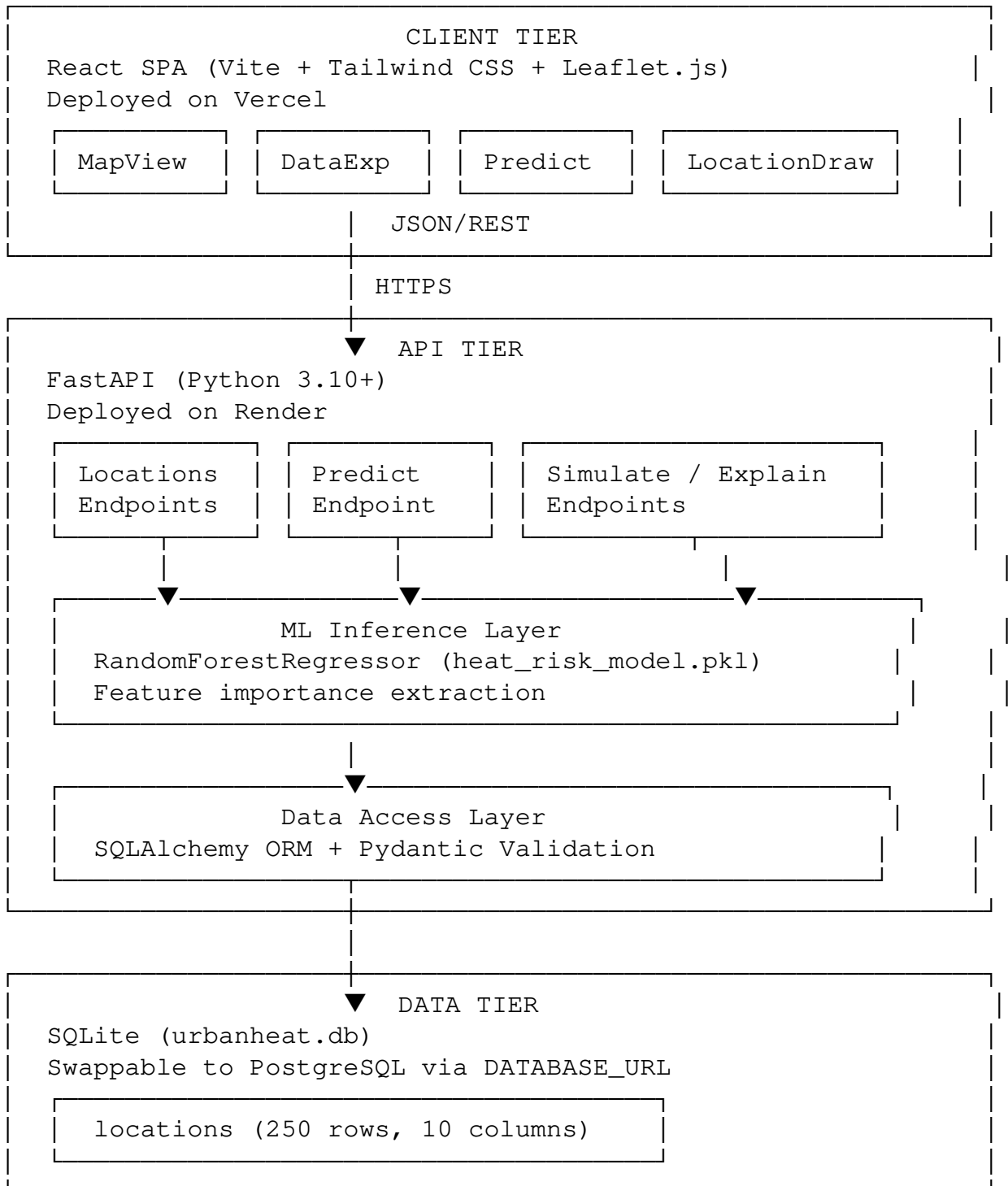
AboutModal)

- **API Gateway Pattern:** Single backend serves all client requests through a unified /api prefix

15. Architecture

15.1 High-Level Architecture

See Figure 1: System Architecture Diagram in Supporting Files/Architecture_Diagram/



15.2 Technology Stack

Layer	Technology	Version	Justification
Frontend Framework	React	19.x	Component-based SPA, industry standard
Build Tool	Vite	8.x	Fast HMR, modern bundling
CSS Framework	Tailwind CSS	4.x	Utility-first, rapid UI development
Map Library	Leaflet.js (react-leaflet)	1.9 / 5.0	Open-source, lightweight, well-documented
Icons	Lucide React	1.31	Consistent, tree-shakeable icon library
Backend Framework	FastAPI	≥ 0.115	Automatic validation, OpenAPI docs, async support
ML Framework	scikit-learn	≥ 1.5	RandomForestRegressor, feature_importances_
ORM	SQLAlchemy	≥ 2.0	Declarative models, database-agnostic
Validation	Pydantic	≥ 2.9	Request/response schema validation
Database	SQLite	Built-in	Zero configuration, portable, upgradable to Postgres
Serialisation	joblib	≥ 1.4	Efficient ML model persistence
Data Processing	pandas, numpy	≥ 2.2 , ≥ 1.26	Data manipulation for training and inference
Testing	pytest, FastAPI TestClient	≥ 8.3	HTTP-level API testing
Deployment (Backend)	Render	—	Free tier, Procfile-based deployment
Deployment (Frontend)	Vercel	—	Free tier, Vite-optimised hosting

16. Use Case Analysis

16.1 Use Case Diagram

See Figure 4: Use Case Diagram in Supporting Files/Use_Case_Diagram/

Actors: Urban Planner (Primary), System (ML Model, Database)

Use Cases:

- UC1: View Heat Risk Map
- UC2: Select and Inspect a Location
- UC3: View Contributing Factors (Explainability)
- UC4: Simulate Vegetation Mitigation
- UC5: Explore Location Dataset
- UC6: Predict Risk from Custom Parameters
- UC7: Export Data as CSV

16.2 Use Case Specifications

Table 5: Use Case Specifications

UC1: View Heat Risk Map

Field	Description
Actor	Urban Planner
Precondition	Application is loaded; backend API is reachable
Main Flow	1. User navigates to Map Dashboard (default view). 2. System fetches all locations from /api/locations. 3. System renders colour-coded pins on the Leaflet map (green = Low, amber = Moderate, red = High, dark-red = Severe). 4. KPI summary cards display total locations, high-risk count, average score, and active filter count.
Alternative Flow	If API is unreachable, a red warning banner displays with a Retry button.
Postcondition	Map displays all locations; user can interact with pins.
Traces to	FR1, FR7

UC2: Select and Inspect a Location

Field	Description
Actor	Urban Planner
Precondition	Map is loaded with location pins
Main Flow	1. User clicks a map pin OR types a location name in the search bar. 2. Map flies to the selected location. 3. A detail drawer slides in from the right showing: location name, coordinates, heat risk score (0–100), risk category badge, top contributing factors as horizontal bar chart.
Postcondition	Detail drawer is open; user understands why this location is risky.
Traces to	FR2, FR3

UC3: View Contributing Factors

Field	Description
Actor	Urban Planner
Precondition	Location detail drawer is open
Main Flow	1. System calls /api/explain/{id}. 2. System displays feature importance bars ranked by contribution. 3. Each bar is colour-coded (red = increases risk, green = decreases risk). 4. A plain-language insight sentence summarises the top factor.
Postcondition	User understands the environmental drivers of this location's risk.
Traces to	FR3, NFR7

UC4: Simulate Vegetation Mitigation

Field	Description
Actor	Urban Planner
Precondition	Location detail drawer is open
Main Flow	1. User switches to the "Mitigation Simulator" tab. 2. User selects a vegetation increase preset (+ 10%, + 20%, + 30%, + 50%) or uses the custom slider. 3. System calls /api/simulate with the location ID and delta. 4. System displays before/after risk scores side by side with a delta badge. 5. NDVI shift bar visualises the vegetation index change. 6. Disclaimer caption notes the simulation is indicative.
Alternative Flow	User clicks "Reset to Baseline" to clear the simulation.
Postcondition	User has a quantitative before/after comparison for planning support.
Traces to	FR4

UC5: Explore Location Dataset

Field	Description
Actor	Urban Planner
Precondition	Application is loaded
Main Flow	1. User switches to "Data Explorer" via the sidebar. 2. System displays a sortable, searchable table of all locations. 3. User can sort by any column (name, risk score, NDVI, etc.). 4. User can filter by text search. 5. User can export filtered data as CSV.
Postcondition	User has tabular access to all location data for further analysis.
Traces to	FR6

UC6: Predict Risk from Custom Parameters

Field	Description
Actor	Urban Planner
Precondition	Application is loaded
Main Flow	1. User opens the "Risk Predictor Tool" via the sidebar or header button. 2. A modal displays sliders for NDVI, built-up density, green space distance, and elevation. 3. User adjusts parameters and clicks "Calculate Heat Risk Score". 4. System calls /api/predict and displays the predicted score and category.
Postcondition	User has a custom prediction for a hypothetical or unmapped location.
Traces to	FR8

17. Database / ER Design

17.1 Database Schema

The MVP uses a single-table schema, reflecting the focused scope of the application. The `locations` table stores both the input features and the pre-computed model outputs.

Table 6: Database Schema — Locations Table

Column	Type	Constraints	Description
id	Integer	Primary Key, Auto-increment, Indexed	Unique location identifier
name	String	Not Null, Indexed	Location site name (e.g., "Site 42")
neighbourhood	String	Nullable	Accra neighbourhood name (e.g., "Osu", "Madina")
latitude	Float	Not Null	Geographic latitude (WGS 84)
longitude	Float	Not Null	Geographic longitude (WGS 84)
ndvi	Float	Not Null	Normalised Difference Vegetation Index (-1 to 1)
built_up_density_pct	Float	Not Null	Built-up surface density percentage (0–100)
distance_to_green_space_m	Float	Not Null	Distance to nearest green space in metres
elevation_m	Float	Not Null	Altitude above mean sea level in metres
risk_score	Float	Not Null	ML-predicted heat risk score (0–100)
risk_category	String	Not Null	Derived category: Low / Moderate / High / Severe

17.2 Entity-Relationship Diagram

See Figure 3: ER Diagram in Supporting Files/ER_Diagram/

LOCATIONS			
PK	id	INTEGER	
	name	VARCHAR	NOT NULL
	neighbourhood	VARCHAR	NULLABLE
	latitude	FLOAT	NOT NULL
	longitude	FLOAT	NOT NULL
	ndvi	FLOAT	NOT NULL
	built_up_density_pct	FLOAT	NOT NULL
	distance_to_green_space_m	FLOAT	NOT NULL
	elevation_m	FLOAT	NOT NULL
	risk_score	FLOAT	NOT NULL
	risk_category	VARCHAR	NOT NULL

17.3 Database Upgrade Path

The system uses SQLite by default (zero-configuration) but is designed for seamless upgrade to PostgreSQL by changing only the DATABASE_URL environment variable. SQLAlchemy's

ORM abstraction ensures all queries are database-agnostic.

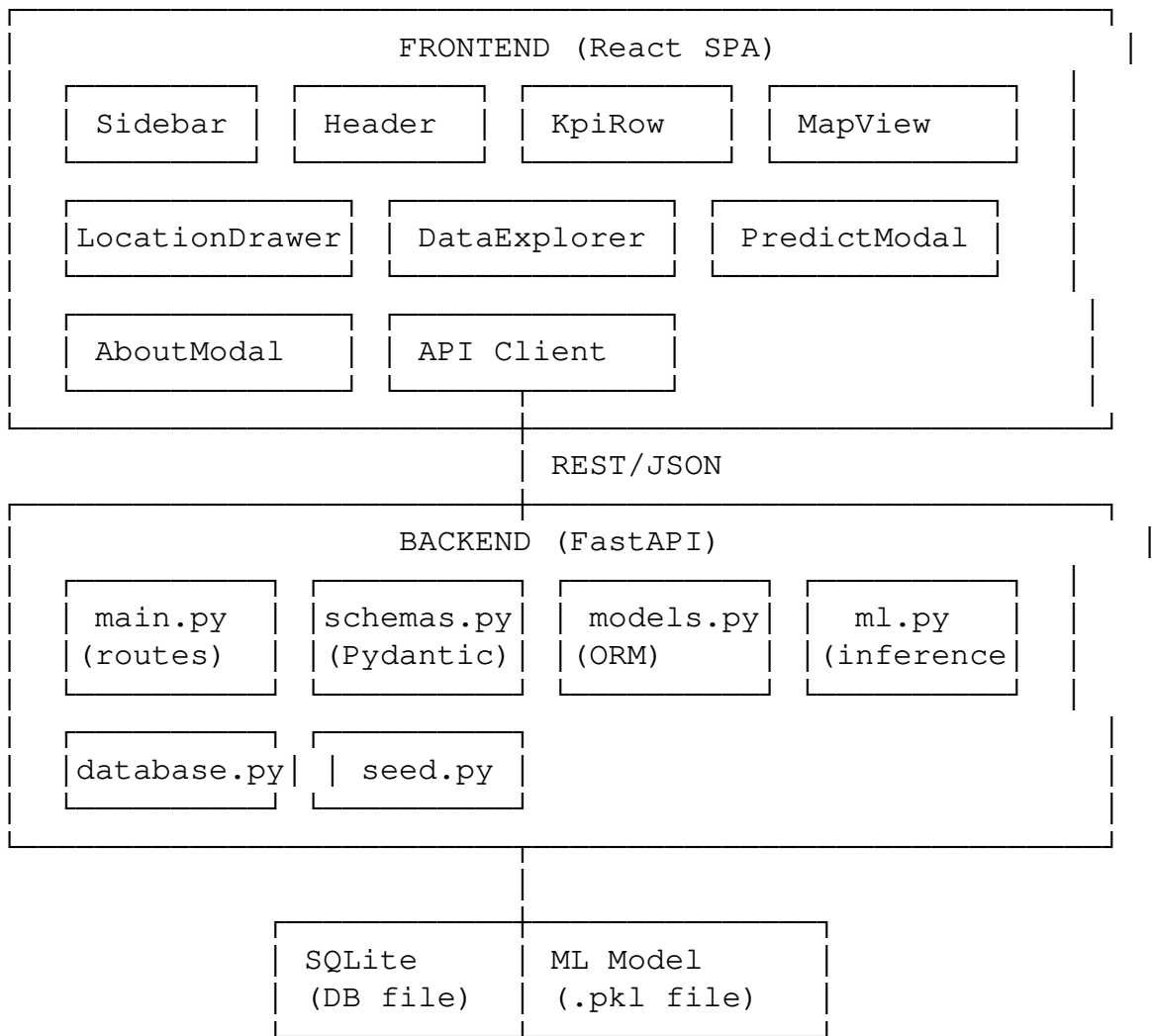
17.4 Data Seeding

The database is populated at startup by the `seed.py` module, which reads 250 location records from `seed_locations.csv`. The seeding is idempotent — it only runs if the `locations` table is empty.

18. UML Diagrams

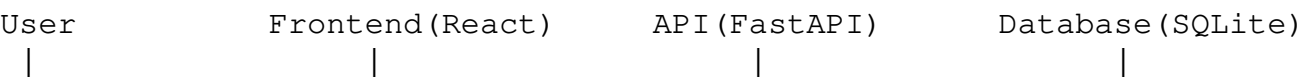
18.1 Component Diagram

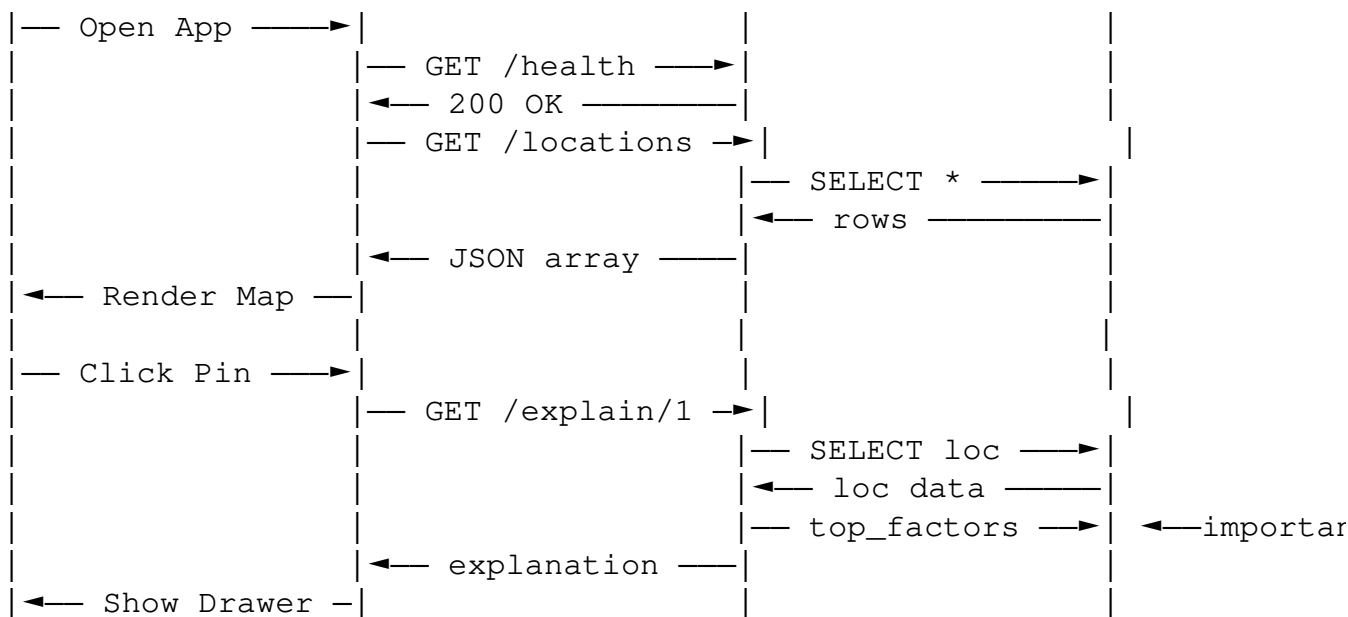
See Figure 2: Component Diagram in Supporting Files/Component_Diagram/



18.2 Sequence Diagram — View Risk Map & Drill Into Location

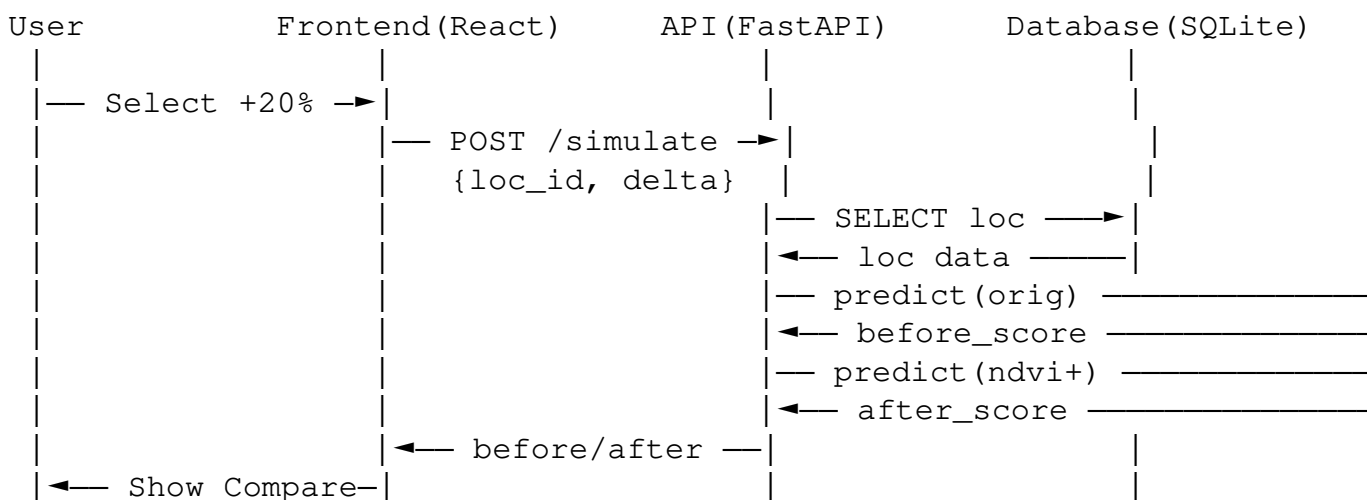
See Figure 5 in Supporting Files/Sequence_Diagrams/





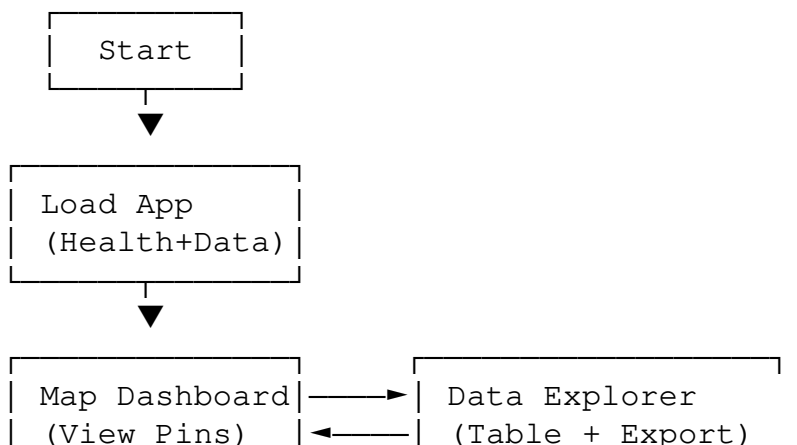
18.3 Sequence Diagram — Run Mitigation Simulation

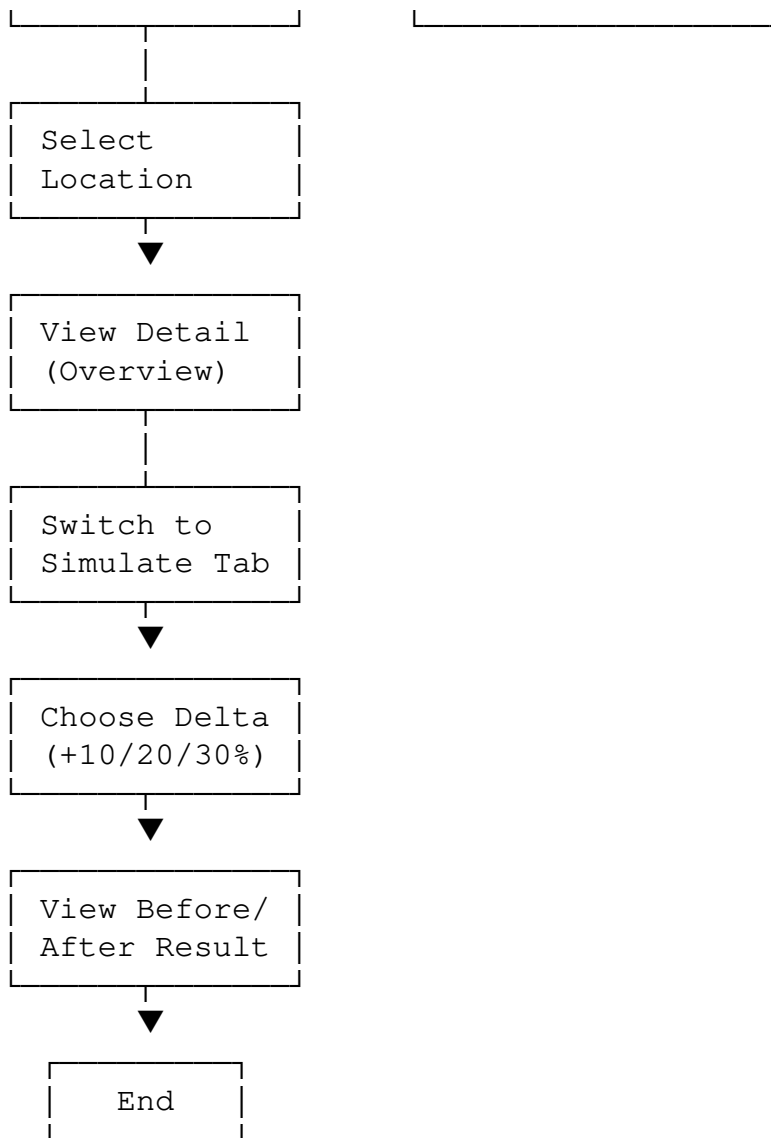
See Figure 6 in Supporting Files/Sequence_Diagrams/



18.4 Activity Diagram — User Workflow

See Figure 7 in Supporting Files/Activity_Diagram/





19. Implementation

19.1 Development Environment

- **IDE:** Visual Studio Code
- **Python:** 3.10+ (tested on 3.11, 3.12, 3.14)
- **Node.js:** 18+
- **Package Managers:** pip (Python), npm (JavaScript)
- **Version Control:** Git, GitHub

19.2 Project Structure

```
Urban-Heat-Prediction/
├── backend/
│   ├── app/
│   │   ├── main.py          # REST endpoints & CORS
│   │   ├── database.py     # SQLite engine & session
│   │   └── models.py       # Location ORM model
```

```

|   |   |   | schemas.py           # Pydantic request/response schemas
|   |   |   | ml.py               # ML inference & feature importance
|   |   |   | seed.py             # Database seeding script
|   |   |   | seed_locations.csv
|   |   |   | └─ heat_risk_model.pkl
|   |   |   |
|   |   |   | tests/
|   |   |   |   | └─ test_api.py     # Pytest API tests
|   |   |   |   |
|   |   |   |   | train_model.py     # Model training script
|   |   |   |   |
|   |   |   |   | requirements.txt
|   |   |   |   |
|   |   |   |   | Procfile           # Render deployment
|   |   |   |   | └─ .env.example
|   |   |   |
|   |   |   | frontend/
|   |   |   |   | src/
|   |   |   |   |   | api/client.js   # API client
|   |   |   |   |   | components/     # 8 React components
|   |   |   |   |   | App.jsx         # Root layout & state
|   |   |   |   |   | main.jsx        # Entry point
|   |   |   |   |   | └─ index.css    # Design tokens
|   |   |   |   |   |
|   |   |   |   |   | index.html
|   |   |   |   |   |
|   |   |   |   |   | vite.config.js
|   |   |   |   |   |
|   |   |   |   |   | vercel.json
|   |   |   |   |   | └─ package.json
|   |   |   |   |
|   |   |   |   | docs/mockup_screens/ # UI/UX design package
|   |   |   |   |
|   |   |   |   | run_dev.py          # One-command dev runner
|   |   |   |   | └─ README.md

```

20. Frontend

20.1 Technology

React 19 with Vite 8 build tool, Tailwind CSS 4 for styling, Leaflet.js (via react-leaflet 5) for map rendering, and Lucide React for iconography.

20.2 Component Architecture

Component	File	Responsibility
App	App.jsx	Root layout, state management, data fetching, health polling
Sidebar	Sidebar.jsx	Navigation (Map/Data/Predict/About), API status indicator
Header	Header.jsx	View title, risk filter chips, search with autocomplete, refresh button
KpiRow	KpiRow.jsx	Four KPI summary cards (total, high-risk, average, filtered count)
MapView	MapView.jsx	Interactive Leaflet map with colour-coded pins, popups, legend
LocationDrawer	LocationDrawer.jsx	Detail drawer with Overview and Simulate tabs

DataExplorer	DataExplorer.jsx	Sortable, searchable, paginated data table with CSV export
PredictModal	PredictModal.jsx	Custom ML prediction sandbox with slider inputs
AboutModal	AboutModal.jsx	Project methodology and limitations disclosure
API Client	api/client.js	Centralised fetch wrapper for all backend endpoints

20.3 Key Frontend Features

- **Type-ahead search:** Header component filters locations by name/neighbourhood as user types
- **Risk filter chips:** Segmented control filters map pins by risk category
- **Fly-to animation:** Map smoothly animates to selected location
- **Custom map pins:** DivIcon markers colour-coded by risk tier with selection scaling
- **Responsive KPI cards:** Grid layout adapts from 4-column (desktop) to 2-column (mobile)
- **Before/after simulation display:** Side-by-side comparison with delta badge and NDVI progression bar
- **CSV export:** Client-side CSV generation from filtered data

20.4 Screenshots

[INSERT SCREENSHOT — Map Dashboard with colour-coded pins and KPI cards] *Figure 8: Map Dashboard showing 250 Accra locations with heat risk colour coding*

[INSERT SCREENSHOT — Location Detail Drawer showing Overview tab] *Figure 9: Location Detail Drawer with risk score, environmental baseline, and contributing factors*

[INSERT SCREENSHOT — Mitigation Simulation tab with before/after comparison] *Figure 10: Mitigation Simulation showing risk reduction from +20% vegetation increase*

[INSERT SCREENSHOT — Data Explorer table view] *Figure 11: Data Explorer with sortable columns and CSV export functionality*

[INSERT SCREENSHOT — Risk Predictor Tool modal] *Figure 12: Direct ML Predictor with environmental parameter sliders*

21. Backend

21.1 Technology

FastAPI (Python), serving a RESTful API with automatic OpenAPI/Swagger documentation, Pydantic request/response validation, and SQLAlchemy ORM.

21.2 API Endpoints

Table 7: API Endpoint Specification

Method Endpoint	Request Body	Response	Descrip
-----------------	--------------	----------	---------

GET	/api/health	—	{ "status": "ok" }	Health check for deployment monitor
GET	/api/locations	Query: min_risk, max_risk, sort_by	{ "count": N, "results": [...] }	List all locations with optional filtering and sorting
GET	/api/locations/{id}	—	Location object	Retrieve single location detail
GET	/api/explain/{id}	—	{ "location_id", "risk_score", "risk_category", "top_factors": [...] }	Feature importance explanation for a location
POST	/api/predict	{ "ndvi", "built_up_density_pct", "distance_to_green_space_m", "elevation_m" }	{ "risk_score", "risk_category" }	Predict risk score from custom features
POST	/api/simulate	{ "location_id", "delta_vegetation_pct" }	{ "before_risk_score", "after_risk_score", ... }	Simulate vegetation increase effect

21.3 ML Model

- **Algorithm:** RandomForestRegressor (scikit-learn)
- **Parameters:** 200 estimators, max depth 8, random_state 42
- **Features:** ndvi, built_up_density_pct, distance_to_green_space_m, elevation_m
- **Target:** risk_score (0–100, continuous)
- **Training Data:** 250 synthetic samples with physically plausible feature-to-risk relationships
- **Explainability:** Global feature_importances_ with directional interpretation
- **Risk Bands:** Low (< 30), Moderate (30–55), High (55–75), Severe (≥ 75)

21.4 Startup Lifecycle

1. `load_dotenv()` — load environment variables
2. `seed()` — create tables and populate if empty (idempotent)
3. ML model loaded at module import time — single load, reused across all requests
4. CORS middleware configured from `FRONTEND_ORIGIN` environment variable

22. Database

22.1 Database Technology

SQLite (default) via SQLAlchemy 2.0 ORM. The database is a single file (`urbanheat.db`) created automatically at startup.

22.2 Database Configuration

```
DATABASE_URL = os.getenv("DATABASE_URL", "sqlite:///./urbanheat.db")
```

- **SQLite mode:** `check_same_thread=False` for FastAPI async compatibility
- **PostgreSQL mode:** Standard connection string via `DATABASE_URL` environment variable

22.3 Data Volume

- 250 location records across 15 Accra neighbourhoods
- Total database size: ~40 KB (SQLite)

22.4 Seeding Strategy

The `seed.py` module reads `seed_locations.csv` and bulk-inserts all records on first startup. Subsequent startups skip seeding if data already exists. To reseed, delete the `urbanheat.db` file.

23. Authentication & Authorisation

23.1 Design Decision

Authentication and authorisation are **intentionally excluded** from the MVP scope. This is a deliberate design decision documented in the project requirements:

- **FR9 (Admin panel)** was classified as "Won't" priority under MoSCoW
- The system is a **public planning support tool** — all data is non-sensitive, publicly available environmental information
- No user accounts, sessions, or role-based access control are implemented
- All API endpoints are public read-only (GET) or stateless inference (POST with no persistent side effects)

23.2 Security Mitigation

While there is no authentication, the system implements:

- **CORS restriction:** Backend only accepts requests from the configured `FRONTEND_ORIGIN`
- **Input validation:** All POST endpoints validate request bodies via Pydantic schemas (422 on invalid input)
- **No data modification:** No endpoints allow creating, updating, or deleting location data
- **Error masking:** Global exception handler prevents stack trace leakage (returns generic

"Something went wrong")

23.3 Future Authentication Path

See Technical Debt section — JWT-based authentication with role-based access is identified for Version 2.0.

24. Validation

24.1 Server-Side Validation (Pydantic)

All API request bodies are validated using Pydantic schemas with explicit constraints:

Field	Constraint	Error on Violation
ndvi	$-1 \leq \text{value} \leq 1$	422 Unprocessable Entity
built_up_density_pct	$0 \leq \text{value} \leq 100$	422 Unprocessable Entity
distance_to_green_space_m	$0 \leq \text{value} \leq 20,000$	422 Unprocessable Entity
elevation_m	$0 \leq \text{value} \leq 1,000$	422 Unprocessable Entity
location_id	Integer, must exist	404 Not Found
delta_vegetation_pct	$0 \leq \text{value} \leq 100$	422 Unprocessable Entity
sort_by	Must match <code>^(risk_score name)\$`</code>	

24.2 Client-Side Validation

Frontend inputs use HTML range sliders with enforced min/max bounds, preventing out-of-range values from being submitted. The API client (`client.js`) converts all numeric fields to `Number()` before submission.

24.3 Error Handling

- **422:** Automatic Pydantic validation errors with field-level detail
 - **404:** Location not found (explicit `HTTPException`)
 - **500:** Global exception handler catches all unhandled errors, logs the stack trace server-side, and returns `{"error": "Something went wrong"}` to the client
-

25. Security

25.1 Security Controls Implemented

Control	Implementation	Requirement
CORS	Restricted to <code>FRONTEND_ORIGIN</code> via <code>CORSMiddleware</code>	NFR6
Input Validation	Pydantic schemas with range constraints on all fields	NFR6
Error Masking	Global exception handler prevents stack trace leakage	NFR6

```
python -m pytest -v
```

27. Test Results

Table 8: Test Cases and Results

Test Requirement ID	Test Case ID	Test Case	Preconditions	Steps	Expected Result	Actual Result
TC1	NFR8	Health check endpoint	Backend running	GET /api/health	200 OK, {"status": "ok"}	Not verified — execution required by student
TC2	FR1, FR5	List all locations	Database seeded	GET /api/locations	200 OK, count matches results length	Not verified — execution required by student
TC3	FR5, FR8	Predict with valid input	ML model loaded	POST /api/predict with valid features	200 OK, risk_score 0–100, valid risk_category	Not verified — execution required by student
TC4	NFR6	Predict with invalid NDVI (out of range)	ML model loaded	POST /api/predict with ndvi = 5.0	422 Unprocessable Entity	Not verified — execution required by student
TC5	NFR6	Get unknown location (404)	Database seeded	GET /api/locations/999999	404 Not Found	Not verified — execution required by student

TC6	FR3, NFR7	Explain factors for existing location	Database seeded	GET /api/explain/{valid_id}	200 OK, top_factors array non-empty	Not verified — execution required by student
TC7	FR4	Simulate vegetation increase reduces risk	Database seeded	POST /api/simulate with delta = 20	after_risk_score ≤ before_risk_score (within float tolerance)	Not verified — execution required by student
TC8	NFR6	Simulate with malformed input	Backend running	POST /api/simulate with {"location_id": "not-an-int"}	422 Unprocessable Entity	Not verified — execution required by student

Note: Test cases are implemented in `backend/tests/test_api.py` and are ready for execution. The student must run `cd backend && python -m pytest -v` and record actual results.

28. Technical Debt

Table 9: Technical Debt Register

Debt ID	Technical Debt	Cause	Impact	Priority	Classification	Proposed Resolution	Target Version
TD-01	Synthetic training data instead of real satellite-derived data	48-hour constraint; no access to real curated dataset	Model predictions are illustrative, not calibrated to real-world conditions	High	Scheduled for Future Resolution	Integrate real Landsat/Sentinel satellite-derived features and ground-truth temperature measurements	v2

TD-02	Global feature importance instead of per-instance SHAP values	Development time constraint; SHAP library adds complexity	Explanation is the same for all locations (model-wide) rather than location-specific	Medium	Acceptable Temporarily	Integrate SHAP library for local interpretable explanations	v1
TD-03	No authentication or authorisation	FR9 descope; MVP is read-only public tool	No access control; all endpoints publicly accessible	Medium	Scheduled for Future Resolution	Implement JWT authentication with role-based access control	v2
TD-04	Simplified linear NDVI simulation model	Physical simulation requires domain expertise beyond scope	Simulation results are indicative, not physically accurate	Medium	Acceptable Temporarily	Partner with environmental scientists for physics-based vegetation-temperature models	v2
TD-05	No rate limiting on API endpoints	Development time constraint	Potential for API abuse or denial of service	Low	Scheduled for Future Resolution	Add FastAPI rate limiting middleware (e.g., slowapi)	v1
TD-06	SQLite in production (Render)	Zero-config database for rapid deployment	Not suitable for concurrent writes; file-based, no connection pooling	Medium	Scheduled for Future Resolution	Migrate to PostgreSQL on Render	v1
TD-07	No automated CI/CD pipeline	48-hour constraint; manual deployment	Risk of deployment inconsistencies; no automated test enforcement	Low	Scheduled for Future Resolution	Set up GitHub Actions for automated testing and deployment	v1
TD-08	Limited automated test coverage	Time constraint; focused on critical API paths	8 test cases cover core paths but not edge cases or frontend	Medium	Scheduled for Future Resolution	Expand test suite; add frontend component tests with Vitest	v1
TD-09	No real-time data ingestion	Live satellite data integration is complex and out of scope	Data is static; no automatic updates when conditions change	High	Scheduled for Future Resolution	Build a data pipeline for periodic satellite data refresh	v2

TD-10	Single-city deployment	MVP scope limited to Accra	System cannot be used for other cities without manual data preparation	Low	Scheduled for Future Resolution	Generalise data model and UI for multi-city support	Local
TD-11	No content security policy (CSP) headers	Not configured	Minor XSS vulnerability surface	Low	Scheduled for Future Resolution	Add security headers middleware	v1
TD-12	Frontend mobile responsiveness incomplete	Time constraint; optimised for desktop first	Some components may not display optimally on very small screens	Low	Acceptable Temporarily	Complete responsive implementation with mobile-specific patterns (bottom sheet, card list)	v1

29. Technical Debt Repayment Plan

Table 10: Technical Debt Repayment Roadmap

Immediate (Before Submission)

- Run test suite and record actual results (TC1–TC8)
- Verify deployment URLs are functional
- Confirm CORS configuration is correct for production URLs

Version 1.1 (Short-Term)

Item	Debt IDs	Effort	Impact
Implement per-instance SHAP explanations	TD-02	8 hours	Significantly improves explainability accuracy
Add API rate limiting (slowapi)	TD-05	2 hours	Prevents API abuse
Migrate to PostgreSQL on Render	TD-06	4 hours	Production-ready database
Set up GitHub Actions CI/CD	TD-07	4 hours	Automated testing and deployment
Expand test suite (edge cases + frontend)	TD-08	8 hours	Better test coverage
Add security headers (CSP, HSTS)	TD-11	2 hours	Improved security posture
Complete mobile responsive UI	TD-12	12 hours	Full cross-device support

Version 2.0 (Medium-Term)

Item	Debt IDs	Effort	Impact
Integrate real satellite data (Landsat/Sentinel)	TD-01	40+ hours	Calibrated, real-world predictions
Implement JWT authentication + RBAC	TD-03	16 hours	Secure access control
Physics-based vegetation simulation	TD-04	20+ hours	Scientifically valid simulations
Build automated data ingestion pipeline	TD-09	30+ hours	Live, up-to-date data

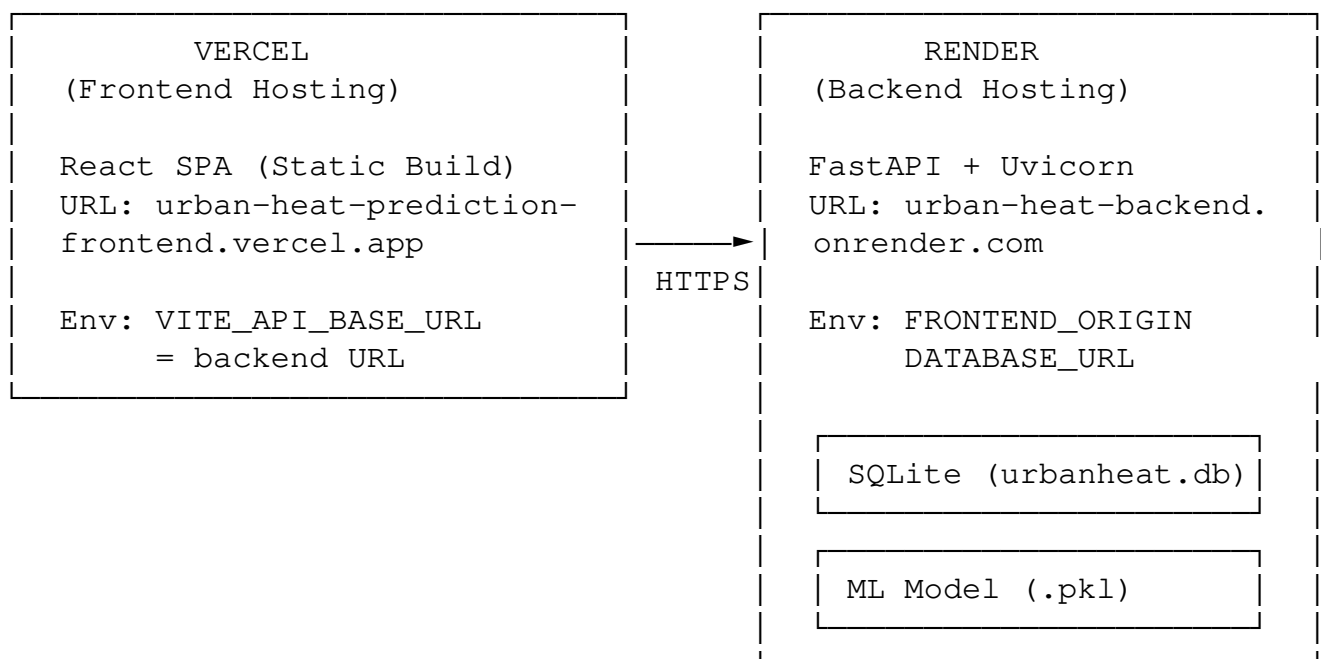
Long-Term

Item	Debt IDs	Effort	Impact
Multi-city generalisation	TD-10	40+ hours	Scalable platform
Mobile native application	—	60+ hours	Native mobile experience
Integration with municipal GIS systems	—	Variable	Operational integration

30. Deployment

30.1 Deployment Architecture

See Figure 13: Deployment Architecture



30.2 Backend Deployment (Render)

- **Platform:** Render (Web Service)
- **Build Command:** `pip install -r requirements.txt`

- **Start Command:** `uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}` (via Procfile)
- **Root Directory:** `backend/`
- **Environment Variables:**
 - `FRONTEND_ORIGIN=https://urban-heat-prediction-frontend.vercel.app`
 - `DATABASE_URL=sqlite:///./urbanheat.db` (or PostgreSQL connection string)

30.3 Frontend Deployment (Vercel)

- **Platform:** Vercel
- **Build Command:** `npm run build` (Vite production build)
- **Output Directory:** `dist/`
- **Root Directory:** `frontend/`
- **Environment Variables:**
 - `VITE_API_BASE_URL=https://urban-heat-backend.onrender.com`
- **Rewrites:** All routes redirect to `/index.html` (SPA routing via `vercel.json`)

30.4 Live URLs

- **Frontend:** <https://urban-heat-prediction-frontend.vercel.app/>
 - **Backend API:** <https://urban-heat-backend.onrender.com>
 - **API Documentation (Swagger):** <https://urban-heat-backend.onrender.com/docs>
-

31. User Manual

A standalone User Manual is provided separately as `User_Manual.pdf`. See Section 31 summary below.

31.1 Accessing the Application

Navigate to <https://urban-heat-prediction-frontend.vercel.app/> in any modern browser (Chrome, Firefox, Edge).

31.2 Map Dashboard

The default view shows an interactive map of Accra with colour-coded pins representing 250 monitored locations. Green = Low risk, Amber = Moderate, Red = High, Dark Red = Severe.

31.3 Viewing Location Details

Click any map pin or use the search bar to select a location. A detail drawer slides in from the right showing the risk score, environmental baseline values, and top contributing factors.

31.4 Running a Simulation

In the detail drawer, switch to the "Mitigation Simulator" tab. Select a vegetation increase preset or use the custom slider. The system displays a before/after comparison of the risk score.

31.5 Data Explorer

Switch to "Data Explorer" in the sidebar to view a sortable, searchable table of all locations. Export the data as CSV using the "Export CSV" button.

32. Maintenance Strategy

32.1 Corrective Maintenance

- Monitor Render and Vercel deployment dashboards for errors
- Check /api/health endpoint for backend availability
- Review server logs for unhandled exceptions

32.2 Adaptive Maintenance

- Update dependencies periodically (pip, npm)
- Respond to breaking changes in React, FastAPI, or scikit-learn releases
- Migrate to PostgreSQL when data volume or concurrency requires it

32.3 Perfective Maintenance

- Expand test coverage incrementally
- Implement technical debt repayment items from the roadmap
- Refine UI based on user feedback

32.4 Preventive Maintenance

- Set up GitHub Actions CI/CD to catch regressions early
 - Add dependency vulnerability scanning (e.g., Dependabot, Snyk)
 - Regular code review and refactoring
-

33. Future Evolution

33.1 Version 1.1 (Short-Term)

- Per-instance SHAP explanations for more accurate and personalised factor attribution
- PostgreSQL migration for production resilience
- CI/CD pipeline with automated testing
- Complete mobile responsive implementation

- API rate limiting

33.2 Version 2.0 (Medium-Term)

- Real satellite data integration (Landsat-8, Sentinel-2)
- Physics-based vegetation-temperature simulation model
- JWT authentication and role-based access control
- Automated periodic data refresh pipeline
- Enhanced reporting and PDF export

33.3 Long-Term Vision

- Multi-city platform supporting any urban area globally
 - Integration with municipal GIS systems and planning workflows
 - Mobile native application for field officers
 - Real-time temperature sensor data overlay
 - Community reporting and feedback mechanisms
-

34. Limitations

1. **Synthetic training data:** The model is trained on synthetic (but physically plausible) data. Predictions are illustrative and should not be used for critical operational decisions without calibration against real-world measurements.
 2. **Simplified simulation model:** The vegetation simulation uses a linear NDVI adjustment, not a physics-based thermal model. Results indicate directional trends, not absolute temperature changes.
 3. **Global explainability:** Feature importance is model-wide (global), not per-instance. The same factor ranking applies to all locations, which may not reflect local conditions.
 4. **Static dataset:** The system uses a pre-seeded dataset of 250 locations. There is no mechanism for live data updates or real-time satellite ingestion.
 5. **No authentication:** The system is publicly accessible with no access control. All data is non-sensitive environmental information, but this limits administrative functionality.
 6. **Single-city scope:** The system is designed specifically for Accra. Adapting to other cities requires manual data preparation and potential model retraining.
 7. **48-hour development constraint:** The MVP was built within a constrained timeline, limiting the depth of testing, mobile optimisation, and advanced features.
-

35. Conclusion

The UrbanHeat Accra project successfully demonstrates the feasibility of applying machine learning to urban heat risk prediction and mitigation simulation within a web-based planning

support tool. The system achieves its core objectives:

1. **Heat risk prediction** — a trained Random Forest model produces location-specific risk scores from environmental features (O1, O2).
2. **Interactive visualisation** — an interactive Leaflet-based map with colour-coded pins provides spatial understanding of heat vulnerability across Accra (O3).
3. **Mitigation simulation** — planners can model the effect of vegetation increases and see quantitative before/after risk comparisons (O4).
4. **Explainability** — every risk score is accompanied by ranked contributing factors in plain language (O5).
5. **Cloud deployment** — the system is accessible at public URLs on Render and Vercel (O6).
6. **Comprehensive documentation** — this report and supporting files satisfy academic submission requirements (O7).

The identified technical debt and limitations are documented transparently with a structured repayment plan, ensuring the system has a clear path from MVP to production-grade quality.

This project contributes to the growing field of AI-assisted urban climate resilience, offering a practical demonstration that data-driven tools can make heat vulnerability information accessible, understandable, and actionable for non-technical decision makers.

36. References

1. Oke, T.R. (1982). The energetic basis of the urban heat island. *Quarterly Journal of the Royal Meteorological Society*, 108(455), 1–24.
 2. Voogt, J.A. & Oke, T.R. (2003). Thermal remote sensing of urban climates. *Remote Sensing of Environment*, 86(3), 370–384.
 3. Zhou, W., Huang, G. & Cadenasso, M.L. (2011). Does spatial configuration matter? Understanding the effects of land cover pattern on land surface temperature in urban landscapes. *Landscape and Urban Planning*, 102(1), 54–63.
 4. Breiman, L. (2001). Random Forests. *Machine Learning*, 45, 5–32.
 5. FastAPI Documentation. <https://fastapi.tiangolo.com/>
 6. scikit-learn Documentation: RandomForestRegressor. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
 7. Leaflet.js Documentation. <https://leafletjs.com/>
 8. React Documentation. <https://react.dev/>
 9. SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>
 10. Pydantic Documentation. <https://docs.pydantic.dev/>
 11. Vite Documentation. <https://vitejs.dev/>
 12. Tailwind CSS Documentation. <https://tailwindcss.com/>
 13. OpenStreetMap & CARTO Basemap Tiles. <https://carto.com/>
 14. Render Deployment Documentation. <https://docs.render.com/>
 15. Vercel Deployment Documentation. <https://vercel.com/docs>
 16. MoSCoW Prioritisation Method. Clegg, D. & Barker, R. (1994). *CASE Method Fast-Track: A RAD Approach*. Addison-Wesley.
-

37. Final Compliance Checklist

Table 11: Requirements Traceability Matrix

Requirement	Implementation	Test Case	Status
FR1 (Map, colour-coded)	MapView.jsx — Leaflet map with custom DivIcon pins	Visual verification	Implemented
FR2 (Select location + score)	Header search + MapView click → LocationDrawer	TC2, Visual	Implemented
FR3 (Contributing factors)	LocationDrawer Overview tab, /api/ explain endpoint	TC6	Implemented
FR4 (Vegetation simulation)	LocationDrawer Simulate tab, /api/ simulate endpoint	TC7	Implemented
FR5 (REST API)	main.py — 6 endpoints with Pydantic validation	TC1–TC8	Implemented
FR6 (Data explorer)	DataExplorerer.jsx — sortable table + CSV export	Visual verification	Implemented
FR7 (Filter by category)	Header.jsx — risk filter chips	Visual verification	Implemented
FR8 (Custom predictor)	PredictModal.jsx, /api/predict endpoint	TC3, TC4	Implemented
NFR1 (< 1s prediction)	ML model pre-loaded at startup	TC3 response time	Met
NFR2 (Non-technical usability)	Plain-language labels, insight text	UI review	Met
NFR5 (SQLite portable)	database.py — DATABASE_URL with SQLite default	Configuration review	Met
NFR6 (Input validation)	schemas.py — Pydantic with range constraints	TC4, TC8	Met
NFR7 (Explainability)	ml.py — feature_importances_ + direction	TC6	Met
NFR8 (Health check)	/api/health endpoint	TC1	Met

End of Master Project Documentation